

LAPORAN PRAKTIKUM
STRUKTUR DATA
SORTING DAN GUI SORTING



Disusun oleh:

Ghinada Fathanawafa Algma
2511533008

Dosen Pengampu:
Dr. Wahyudi S.T. M.T

Asisten Praktikum:
Sherly Sukmadira

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur atas kehadiran Tuhan Yang Maha Esa, berkat Rahmat dan karunia-Nya laporan praktikum Struktur Data tentang sorting dan GUI Sorting menggunakan bahasa Java ini dapat saya selesaikan dengan baik. Laporan ini saya susun sebagai bentuk pertanggungjawaban atas praktikum yang telah saya laksanakan.

Dalam penyusunan laporan ini saya banyak mendapat bimbingan serta dukungan dari beberapa pihak, oleh karena itu saya mengucapkan terima kasih kepada dosen pengampu, asisten praktikum dan rekan rekan praktikan yang turut membantu dalam pelaksanaan kegiatan praktikum.

Saya menyadari bahwa laporan praktikum ini jauh dari sempurna. Oleh karena itu kritik dan saran yang membangun sangat berguna untuk perkembangan lebih baik kedepannya. Semoga laporan praktikum ini dapat bermanfaat bagi saya serta pembaca, khususnya dalam memahami konsep Sorting.

Padang, Juni 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI	ii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang	1
1.2 Tujuan	1
1.3 Manfaat	1
BAB II PEMBAHASAN	2
2.1 Dasar Teori	2
2.1.1 Shell Sort.....	2
2.1.2 Quick Sort	2
2.1.3 Merge Sort.....	2
2.2 Kode Program	2
2.2.1 Class ShellSort_2511533008	2
2.2.2 Class QuickSort_2511533008.....	5
2.2.3 Class MergeSort_2511533008	7
2.2.4 Class BubbleSortGUI_2511533008.....	9
2.3 Output Program.....	15
2.3.1 Output ShellSort_2511533008.....	15
2.3.2 Output QuickSort_2511533008	16
2.3.3 Output MergeSort_2511533008.....	16
2.3.4 Output BubbleSortGUI_2511533008	16
BAB III PENUTUP	21

3.1 Kesimpulan	21
DAFTAR PUSTAKA.....	22
TUGAS	23
1. Kode Program	23
2. Output Program.....	26

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam dunia pemrograman dan pengolahan data, algoritma sorting memiliki peran yang sangat penting karena memungkinkan data disusun secara teratur sehingga lebih mudah diakses, dianalisis, dan diproses. Sorting tidak hanya digunakan untuk mengurutkan data dalam urutan naik (ascending) atau turun (descending), tetapi juga menjadi dasar bagi berbagai algoritma lain seperti pencarian dan analisis statistik. Melalui praktikum struktur data ini, mahasiswa diharapkan dapat memahami konsep dasar pengurutan, mengenal berbagai jenis algoritma seperti Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, Shell Sort dan Quick Sort, serta mampu menganalisis efisiensi masing-masing algoritma melalui kompleksitas waktu. Dengan adanya praktikum ini, kita tidak hanya belajar mengimplementasikan algoritma sorting dalam bahasa pemrograman, tetapi juga membandingkan performa algoritma terhadap berbagai ukuran data sehingga dapat menentukan metode yang paling sesuai untuk kebutuhan sistem.

1.2 Tujuan

1. Memahami konsep dasar algoritma sorting.
2. Dapat mengimplementasikan algoritma sorting dan membuat GUI nya.

1.3 Manfaat

1. Mengetahui dan memahami konsep dasar algoritma sorting.
2. Bisa membuat program algoritma sorting dengan GUI.

BAB II

PEMBAHASAN

2.1 Dasar Teori

2.1.1 Shell Sort

Running time dari metode Shell Sort bergantung pada pemilihan sequence number-nya. Disarankan untuk memilih sequence number dimulai dari $N/2$, kemudian membaginya lagi dengan 2, seterusnya hingga mencapai 1. Shell sort menggunakan 3 nested loop, untuk merepresentasikan sebuah pengembangan yang substansial terhadap metode insertion sort.

2.1.2 Quick Sort

Quick Sort adalah algoritma pengurutan berbasis *divide and conquer* yang bekerja dengan memilih sebuah **pivot**, lalu membagi data menjadi dua bagian: elemen lebih kecil di kiri dan lebih besar di kanan. Proses ini dilakukan secara **rekursif** hingga seluruh data terurut.

2.1.3 Merge Sort

Merge adalah proses dalam metode *Divide and Conquer* yang menggabungkan dua subarray yang sudah terurut menjadi satu array besar yang juga terurut, dilakukan dengan cara membandingkan elemen dari kedua bagian secara berurutan lalu menempatkan elemen yang lebih kecil terlebih dahulu hingga semua elemen habis; dengan demikian, hasil penggabungan memastikan keseluruhan array tetap terurut.

2.2 Kode Program

2.2.1 Class ShellSort_2511533008

Kode Program:

```
package pekan8_2511533008;

public class ShellSort_2511533008 {

    public static void shellSort_2511533008(int [] A_3008) {
        int n_3008 = A_3008.length;
```

```

        int gap_3008 = n_3008/2;
        while (gap_3008 > 0) {
            for (int i_3008 = gap_3008; i_3008 < n_3008;
i_3008++) {
                int temp_3008 = A_3008[i_3008];
                int j_3008 = i_3008;
                while (j_3008 >= gap_3008 && A_3008[j_3008
- gap_3008] > temp_3008) {
                    A_3008[j_3008] = A_3008[j_3008 -
gap_3008];
                    j_3008 = j_3008 - gap_3008;
                }
                A_3008[j_3008] = temp_3008;
            }
            gap_3008 = gap_3008/2;
        }
    }

    public static void main(String[] args) {
        int[] data_3008 = {3, 10, 4, 6, 8, 9, 7, 2, 1, 5};

        System.out.print("Sebelum: ");
        printArray_3008(data_3008);

        shellSort_2511533008(data_3008);

        System.out.print("Sesudah (shell sort): ");
        printArray_3008(data_3008);

    }

    public static void printArray_3008(int[] arr_3008) {
        for (int i_3008 : arr_3008) System.out.print(i_3008 + "
");
        System.out.println();
    }
}

```

Penjelasan :

1. Method shellSort_2511533008

```

public static void shellSort_2511533008(int [] A_3008) {
    int n_3008 = A_3008.length;
    int gap_3008 = n_3008/2;
    while (gap_3008 > 0) {
        for (int i_3008 = gap_3008; i_3008 < n_3008; i_3008++) {
            int temp_3008 = A_3008[i_3008];
            int j_3008 = i_3008;
            while (j_3008 >= gap_3008 && A_3008[j_3008 - gap_3008] > temp_3008) {
                A_3008[j_3008] = A_3008[j_3008 - gap_3008];
                j_3008 = j_3008 - gap_3008;
            }
            A_3008[j_3008] = temp_3008;
        }
        gap_3008 = gap_3008/2;
    }
}

```

Gambar 2.2.1.1 method shellsort

- `gap_3008 = n_3008/2` → jarak awal setengah panjang array.
- Loop while (`gap_3008 > 0`) → selama gap masih ada, lakukan pengurutan.
- Loop for → mulai dari indeks `gap_3008` hingga akhir array.
- Simpan elemen sementara (`temp_3008`) lalu geser elemen yang lebih besar ke kanan sesuai gap.
- Masukkan kembali elemen pada posisi yang tepat.
- Gap dibagi dua setiap iterasi (`gap_3008 = gap_3008/2`).

2. Method Main (Program Utama)

```

public static void main(String[] args) {
    int[] data_3008 = {3, 10, 4, 6, 8, 9, 7, 2, 1, 5};

    System.out.print("Sebelum: ");
    printArray_3008(data_3008);

    shellSort_2511533008(data_3008);

    System.out.print("Sesudah (shell sort): ");
    printArray_3008(data_3008);
}

```

Gambar 2.2.1.2 method utama

- Membuat array `data_3008` berisi angka acak.
- Cetak array sebelum diurutkan.
- Panggil `shellSort_2511533008` untuk mengurutkan.
- Cetak array setelah diurutkan.

3. Method printArray


```

public static void printArray_3008(int[] arr_3008) {
    for (int i_3008 : arr_3008) System.out.print(i_3008 + " ");
    System.out.println();
}

```

Gambar 2.2.1.3 method print array

Method ini dipanggil untuk menampilkan isi yang ada dalam array.

2.2.2 Class QuickSort_2511533008

Kode Program:

```

package pekan8_2511533008;

public class QuickSort_2511533008 {
    static void swap_3008(int[] arr_3008, int i_3008, int j_3008)
    {
        int temp_3008 = arr_3008[i_3008];
        arr_3008[i_3008] = arr_3008[j_3008];
        arr_3008[j_3008] = temp_3008;
    }
    //metode tambahan untuk mengatur pivot menggunakan median-of-
    three
    static void medianOfThree_3008(int[] arr_3008, int low_3008,
    int high_3008) {
        int mid_3008 = low_3008 + (high_3008 - low_3008)/2;

        //urutkan elemen low, mid, high
        if (arr_3008[low_3008] > arr_3008[mid_3008]) {
            swap_3008(arr_3008, low_3008, mid_3008);
        }
        if (arr_3008[low_3008] > arr_3008[high_3008]) {
            swap_3008(arr_3008, low_3008, high_3008);
        }
        if (arr_3008[mid_3008] > arr_3008[high_3008]) {
            swap_3008(arr_3008, mid_3008, high_3008);
        }
        swap_3008(arr_3008, mid_3008, high_3008);
    }
    static int partition_3008(int[] arr_3008, int low_3008, int
    high_3008) {
        //panggil fungsi medianofthree sebelum menentukan pivot
        medianOfThree_3008(arr_3008, low_3008, high_3008);

        int pivot_3008 = arr_3008[high_3008]; //sekarang
        arr[high] sudah berisi nilai median
        int i_3008 = (low_3008 - 1);

        for (int j_3008 = low_3008; j_3008 <= high_3008 - 1;
        j_3008++) {
            //jika elemen saat ini lebih kecil dari atau
            sama dengan pivot

```

```

        if(arr_3008[j_3008] < pivot_3008) {
            //increment indeks elemen yang lebih kecil
            i_3008++;
            swap_3008(arr_3008, i_3008, j_3008);
        }
    }
    swap_3008(arr_3008, i_3008 + 1, high_3008);
    return (i_3008 + 1);
}

static void quickSort_2511533008(int[] arr_3008, int low_3008,
int high_3008) {
    if(low_3008 < high_3008) {
        int pi_3008 = partition_3008(arr_3008, low_3008,
high_3008);
        quickSort_2511533008(arr_3008, low_3008, pi_3008
- 1);
        quickSort_2511533008(arr_3008, pi_3008 + 1,
high_3008);
    }
}

public static void printArr_3008(int[] arr_3008) {
    for(int i_3008 = 0; i_3008 < arr_3008.length; i_3008++)
    {
        System.out.print(arr_3008[i_3008] + " ");
        System.out.println();
    }
}

public static void main(String[] args) {
    int[] arr_3008 = {10, 7, 8, 9, 1, 5};
    int N_3008 = arr_3008.length;
    System.out.print("Data sebelum diurutkan: ");
    printArr_3008(arr_3008);

    quickSort_2511533008(arr_3008, 0, N_3008 - 1);

    System.out.print("Data terurut quicksort: ");
    printArr_3008(arr_3008);

}
}

```

Penjelasan:

1. swap_3008 → menukar dua elamen dalam array
2. medianOfThree_3008 → memilih pivot dengan cara mengambil median dari tiga elemen (low, mid, high).
3. partition_3008

- a. Memanggil `medianOfThree_3008` agar pivot lebih optimal.
 - b. Menempatkan pivot di posisi akhir (`arr[high]`).
 - c. Menggeser elemen lebih kecil dari pivot ke kiri.
 - d. Mengembalikan indeks posisi pivot setelah partisi.
4. `quicksort_2511533008`
 - a. Jika `low < high`, lakukan partisi.
 - b. Rekursi pada subarray kiri (`low → pi-1`) dan kanan (`pi+1 → high`).
5. `printArr_3008 → print isi array`
6. `main` (program utama)
 - a. Membuat array `{10, 7, 8, 9, 1, 5}`.
 - b. Cetak sebelum diurutkan.
 - c. Jalankan `quickSort_2511533008`.
 - d. Cetak hasil setelah diurutkan.

2.2.3 Class MergeSort_2511533008

Kode Program:

```
package pekan8_2511533008;

public class MergeSort_2511533008 {
    void merge_3008(int arr_3008[], int l_3008, int m_3008, int
r_3008) {
        //find sizes of two subarrays to be merged
        int n1_3008 = m_3008 - l_3008 + 1;
        int n2_3008 = r_3008 - m_3008;
        /* create temp arrays */
        int L_3008[] = new int[n1_3008];
        int R_3008[] = new int[n2_3008];
        /* copy data to temp arrays */
        for (int i_3008 = 0; i_3008 < n1_3008; i_3008++)
            L_3008[i_3008] = arr_3008[l_3008 + i_3008];
        for (int j_3008 = 0; j_3008 < n2_3008; j_3008++)
            R_3008[j_3008] = arr_3008[m_3008 + 1 + j_3008];
        int i_3008 = 0, j_3008 = 0;
        //initial index of merged subarray array
        int k_3008 = l_3008;
        while (i_3008 < n1_3008 && j_3008 < n2_3008) {
            if (L_3008[i_3008] <= R_3008[j_3008]) {
```

```

        arr_3008[k_3008] = L_3008[i_3008];
        i_3008++;
    } else {
        arr_3008[k_3008] = R_3008[j_3008];
        j_3008++;
    }
    k_3008++;
}
/*copy remaining elements of L[] if any */
while (i_3008 < n1_3008) {
    arr_3008[k_3008] = L_3008[i_3008];
    i_3008++;
    k_3008++;
}
/*copy remaining elements of R[] if any */
while (j_3008 < n2_3008) {
    arr_3008[k_3008] = R_3008[j_3008];
    j_3008++;
    k_3008++;
}
}

void sort_3008(int arr_3008[], int l_3008, int r_3008) {
    if (l_3008 < r_3008) {
        //find the middle point
        int m_3008 = (l_3008 + r_3008)/2;
        //sort first and second halves
        sort_3008(arr_3008, l_3008, m_3008);
        sort_3008(arr_3008, m_3008 + 1, r_3008);
        //merge the sorted halves
        merge_3008(arr_3008, l_3008, m_3008, r_3008);
    }
}

/* A utility function to print array of size n */
static void printArray_3008(int arr_3008[]) {
    int n_3008 = arr_3008.length;
    for (int i_3008 = 0; i_3008 < n_3008; ++i_3008) {
        System.out.print(arr_3008[i_3008] + " ");
    }
    System.out.println();
}

}

public static void main(String[] args) {
    int arr_3008[] = {12, 11, 13, 5, 6, 7};
    System.out.println("Sebelum terurut: ");
    printArray_3008(arr_3008);
    MergeSort_2511533008 ob_3008 = new
MergeSort_2511533008();
    ob_3008.sort_3008(arr_3008, 0, arr_3008.length - 1);
    System.out.println("\nSesudah terurut menggunakan merge
sort: ");
    printArray_3008(arr_3008);
}

```

```
}  
}
```

Penjelasan:

1. merge_3008
 - a. Hitung ukuran dua subarray: kiri (n1_3008) dan kanan (n2_3008).
 - b. Buat array sementara L_3008 dan R_3008 untuk menampung data.
 - c. Bandingkan elemen dari kedua array, masukkan yang lebih kecil ke array utama.
 - d. Jika masih ada sisa elemen di salah satu array, salin semuanya ke array utama.
2. sort_3008
 - a. Jika $l_3008 < r_3008$, cari titik tengah m_3008 .
 - b. Rekursi untuk mengurutkan bagian kiri ($l_3008 \rightarrow m_3008$) dan kanan ($m_3008+1 \rightarrow r_3008$).
 - c. Gabungkan hasil dengan memanggil `merge_3008`.
3. printArray_3008 → cetak isi array
4. main(program utama)
 - a. Array awal: {12, 11, 13, 5, 6, 7}.
 - b. Cetak sebelum diurutkan.
 - c. Panggil `sort_3008` untuk mengurutkan.
 - d. Cetak hasil setelah diurutkan.

2.2.4 Class BubbleSortGUI_2511533008

Kode Program:

```
package pekan8_2511533008;
```

```

import java.awt.BorderLayout;
import java.awt.FlowLayout;
import java.awt.Font;
import java.awt.Color;
import java.awt.Dimension;

import javax.swing.JFrame;
import javax.swing.JPanel;
import javax.swing.JLabel;
import javax.swing.JButton;
import javax.swing.JTextField;
import javax.swing.JTextArea;
import javax.swing.JScrollPane;
import javax.swing.JOptionPane;
import javax.swing.BorderFactory;
import javax.swing.SwingConstants;

public class BubbleSortGUI_2511533008 extends JFrame {

    private static final long serialVersionUID = 1L;

    private int[] array_3008;
    private JLabel[] labelArray_3008;
    private JButton stepButton_3008, resetButton_3008,
setButton_3008;
    private JTextField inputField_3008;
    private JPanel panelArray_3008;
    private JTextArea stepArea_3008;

    private int i_3008 = 0, j_3008;
    private boolean sorting_3008 = false;
    private int stepCount_3008 = 1;

    public BubbleSortGUI_2511533008() {
        setTitle("Bubble Sort: Langkah per Langkah");
        setSize(750, 400);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout());

        // Panel input
        JPanel inputPanel_3008 = new JPanel(new FlowLayout());
        inputField_3008 = new JTextField(30);
        setButton_3008 = new JButton("Set Array");
        inputPanel_3008.add(new JLabel("Masukkan angka (pisahkan
dengan koma):"));
        inputPanel_3008.add(inputField_3008);
        inputPanel_3008.add(setButton_3008);

        // Panel array visual
        panelArray_3008 = new JPanel();
        panelArray_3008.setLayout(new FlowLayout());

```

```

// Panel kontrol
JPanel controlPanel_3008 = new JPanel();
stepButton_3008 = new JButton("Langkah Selanjutnya");
resetButton_3008 = new JButton("Reset");
stepButton_3008.setEnabled(false);
controlPanel_3008.add(stepButton_3008);
controlPanel_3008.add(resetButton_3008);

// Area teks untuk log langkah-langkah
stepArea_3008 = new JTextArea(8, 60);
stepArea_3008.setEditable(false);
stepArea_3008.setFont(new Font("Monospaced", Font.PLAIN,
14));
JScrollPane scrollPane_3008 = new
JScrollPane(stepArea_3008);

// Tambahkan panel ke frame
add(inputPanel_3008, BorderLayout.NORTH);
add(panelArray_3008, BorderLayout.CENTER);
add(controlPanel_3008, BorderLayout.SOUTH);
add(scrollPane_3008, BorderLayout.EAST);

// Event Set Array
setButton_3008.addActionListener(e ->
setArrayFromInput_3008());

// Event Langkah Selanjutnya
stepButton_3008.addActionListener(e -> performStep_3008());

// Event Reset
resetButton_3008.addActionListener(e -> reset_3008());
}

private void setArrayFromInput_3008() {
    String text_3008 = inputField_3008.getText().trim();

    if (text_3008.isEmpty()) return;

    String[] parts_3008 = text_3008.split(",");
    array_3008 = new int[parts_3008.length];

    try {
        for (int k_3008 = 0; k_3008 < parts_3008.length;
k_3008++) {
            array_3008[k_3008] =
Integer.parseInt(parts_3008[k_3008].trim());
        }
    } catch (NumberFormatException e) {
        JOptionPane.showMessageDialog(
            this,
            "Masukkan hanya angka yang dipisahkan dengan
koma",
            "Error",

```

```

JOptionPane.ERROR_MESSAGE
    );
    return;
}

i_3008 = 0;
j_3008 = 0;
stepCount_3008 = 1;
sorting_3008 = true;

stepButton_3008.setEnabled(true);
stepArea_3008.setText("");

panelArray_3008.removeAll();

labelArray_3008 = new JLabel[array_3008.length];

for (int k_3008 = 0; k_3008 < array_3008.length; k_3008++) {
    labelArray_3008[k_3008] = new
JLabel(String.valueOf(array_3008[k_3008]));
    labelArray_3008[k_3008].setFont(new Font("Arial",
Font.BOLD, 24));
    labelArray_3008[k_3008].setOpaque(true);
    labelArray_3008[k_3008].setBackground(Color.WHITE);

labelArray_3008[k_3008].setBorder(BorderFactory.createLineBorder(Col
or.BLACK));
    labelArray_3008[k_3008].setPreferredSize(new
Dimension(50, 50));

labelArray_3008[k_3008].setHorizontalAlignment(SwingConstants.CENTER
);
    panelArray_3008.add(labelArray_3008[k_3008]);
}
panelArray_3008.revalidate();
panelArray_3008.repaint();
}
private void performStep_3008() {
    if (!sorting_3008 || j_3008 >= array_3008.length - 1) {
        sorting_3008 = false;
        stepButton_3008.setEnabled(false);
        JOptionPane.showMessageDialog(this, "Sorting selesai!");
        return;
    }

    resetHighlights_3008();
    StringBuilder stepLog_3008 = new StringBuilder();

    labelArray_3008[j_3008].setBackground(Color.CYAN);
    labelArray_3008[j_3008 + 1].setBackground(Color.CYAN);

    if (array_3008[j_3008] > array_3008[j_3008 + 1]) {
        // Swap

```



```

        int temp = array_3008[j_3008];
        array_3008[j_3008] = array_3008[j_3008 + 1];
        array_3008[j_3008 + 1] = temp;

        labelArray_3008[j_3008].setBackground(Color.RED);
        labelArray_3008[j_3008 + 1].setBackground(Color.RED);

        stepLog_3008.append("Langkah
").append(stepCount_3008).append(": Menukar elemen ke-")
            .append(j_3008).append("
").append(array_3008[j_3008]).append(" dengan ke-")
            .append(j_3008 + 1).append("
").append(array_3008[j_3008 + 1]).append(")\n");
    } else {
        stepLog_3008.append("Langkah
").append(stepCount_3008).append(": Tidak ada pertukaran antara ke-
")
            .append(j_3008).append(" dan ke-").append(j_3008
+ 1).append(")\n");
    }

    stepLog_3008.append("Hasil:
").append(arrayToString_3008(array_3008)).append("\n\n");
    stepArea_3008.append(stepLog_3008.toString());

    updateLabels_3008();
    j_3008++;

    if (j_3008 >= array_3008.length - i_3008 - 1) {
        j_3008 = 0;
        i_3008++;
    }

    stepCount_3008++;

    if (i_3008 >= array_3008.length - 1) {
        sorting_3008 = false;
        stepButton_3008.setEnabled(false);
        JOptionPane.showMessageDialog(this, "Sorting selesai!");
    }
}

private void updateLabels_3008() {
    for (int k_3008 = 0; k_3008 < array_3008.length; k_3008++) {
labelArray_3008[k_3008].setText(String.valueOf(array_3008[k_3008]));
    }
}

private void reset_3008() {
    inputField_3008.setText("");
    panelArray_3008.removeAll();
    panelArray_3008.revalidate();
    panelArray_3008.repaint();
}

```

```

        stepArea_3008.setText("");
        stepButton_3008.setEnabled(false);
        sorting_3008 = false;
        i_3008 = 0;
        j_3008 = 0;
        stepCount_3008 = 1;
    }
    private String arrayToString_3008(int[] arr_3008) {
        StringBuilder sb_3008 = new StringBuilder();
        for (int k_3008 = 0; k_3008 < arr_3008.length; k_3008++) {
            sb_3008.append(arr_3008[k_3008]);
            if (k_3008 < arr_3008.length - 1) {
                sb_3008.append(", ");
            }
        }
        return sb_3008.toString();
    }
    private void resetHighlights_3008() {
        if (labelArray_3008 != null) {
            for (JLabel label : labelArray_3008) {
                label.setBackground(Color.WHITE);
            }
        }
    }
    public static void main(String[] args) {
        javax.swing.SwingUtilities.invokeLater(() -> {
            BubbleSortGUI_2511533008 gui_3008 =
                new BubbleSortGUI_2511533008();
            gui_3008.setVisible(true);
        });
    }
}

```

Penjelasan:

Class BubbleSortGUI_2511533008 → Turunan dari JFrame, berfungsi sebagai jendela utama aplikasi.

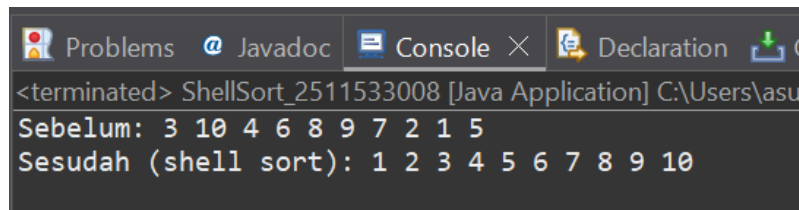
Komponen utama:

- **Input field** (JTextField) → tempat memasukkan array angka.
- **Set Array button** → untuk memproses input menjadi array.
- **Step button** → mengeksekusi satu langkah *Insertion Sort*.
- **Reset button** → menghapus semua data dan mengulang dari awal.
- **Panel array** → menampilkan angka dalam bentuk label.
- **Text area log** → menampilkan catatan langkah-langkah sorting.

1. **setArrayFromInput_3008:**
 - Membaca input angka dari pengguna.
 - Membuat array dan label visual untuk setiap elemen.
 - Menginisialisasi variabel kontrol sorting.
2. **performStep_3008:**
 - Menjalankan satu langkah bubble sort.
 - Menyorot elemen yang dibandingkan (biru).
 - Jika terjadi pertukaran, warnai merah dan lakukan swap.
 - Catat hasil ke stepArea_3008.
 - Update label array agar sesuai dengan nilai terbaru.
 - Mengatur indeks i dan j untuk iterasi bubble sort.
 - Jika selesai, tampilkan pesan “Sorting selesai!”.
3. **reset_3008:** membersihkan input, panel, dan log.
4. **arrayToString_3008:** mengubah array menjadi string untuk ditampilkan di log.
5. **resetHighlights_3008:** mengembalikan warna label ke putih.
6. **Main (program utama):** menjalankan GUI dengan `SwingUtilities.invokeLater`.

2.3 Output Program

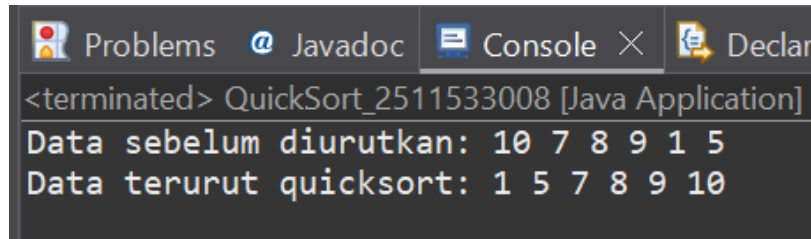
2.3.1 Output ShellSort_2511533008



```
<terminated> ShellSort_2511533008 [Java Application] C:\Users\asus
Sebelum: 3 10 4 6 8 9 7 2 1 5
Sesudah (shell sort): 1 2 3 4 5 6 7 8 9 10
```

Gambar 2.3.1 output shell sort

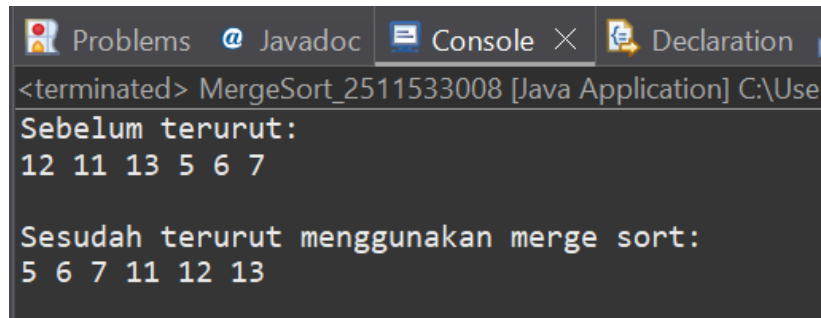
2.3.2 Output QuickSort_2511533008



```
<terminated> QuickSort_2511533008 [Java Application]
Data sebelum diurutkan: 10 7 8 9 1 5
Data terurut quicksort: 1 5 7 8 9 10
```

Gambar 2.3.2 output quick sort

2.3.3 Output MergeSort_2511533008

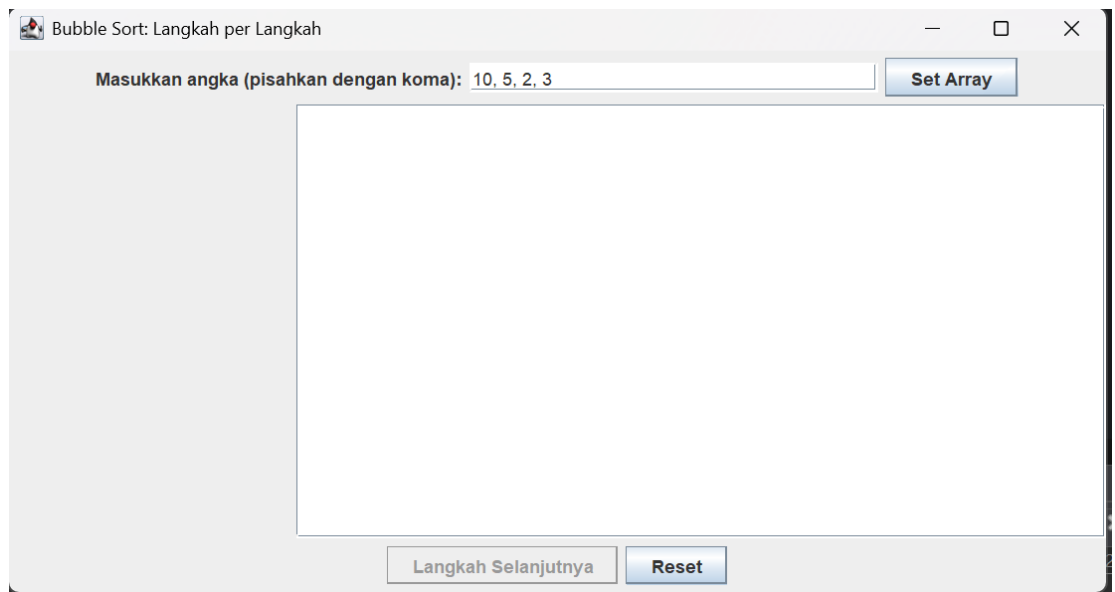


```
<terminated> MergeSort_2511533008 [Java Application] C:\Use
Sebelum terurut:
12 11 13 5 6 7

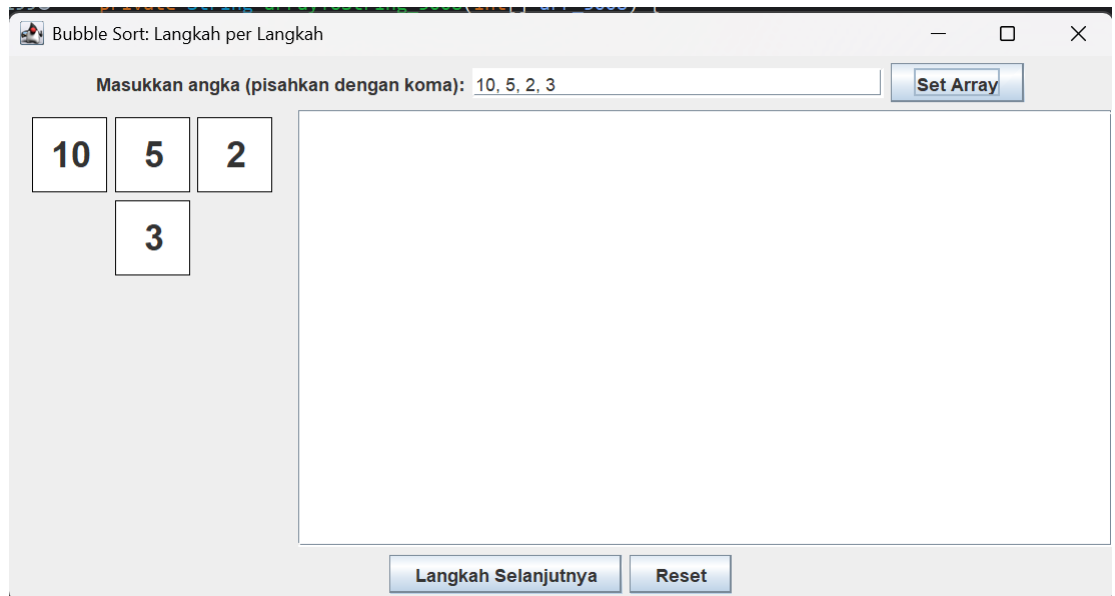
Sesudah terurut menggunakan merge sort:
5 6 7 11 12 13
```

Gambar 2.3.3 output merge sort

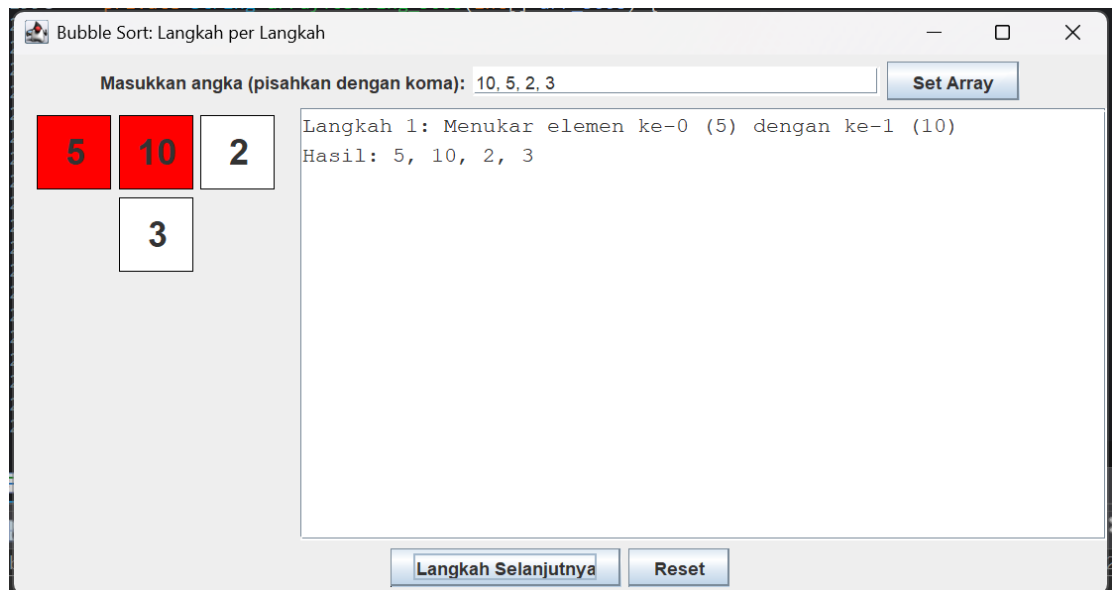
2.3.4 Output BubbleSortGUI_2511533008



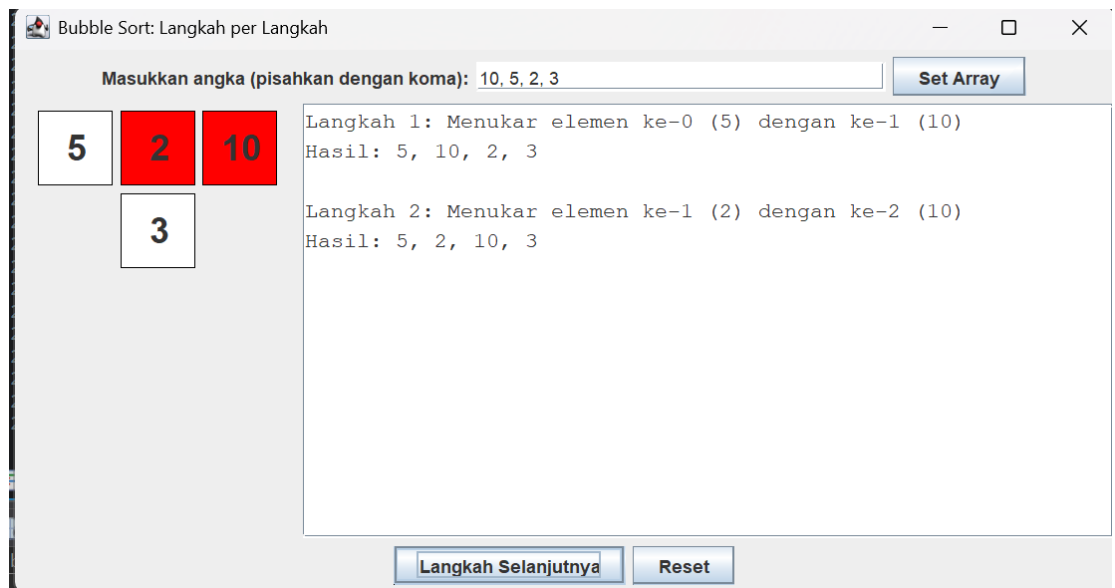
Gambar 2.3.4.1 input angka



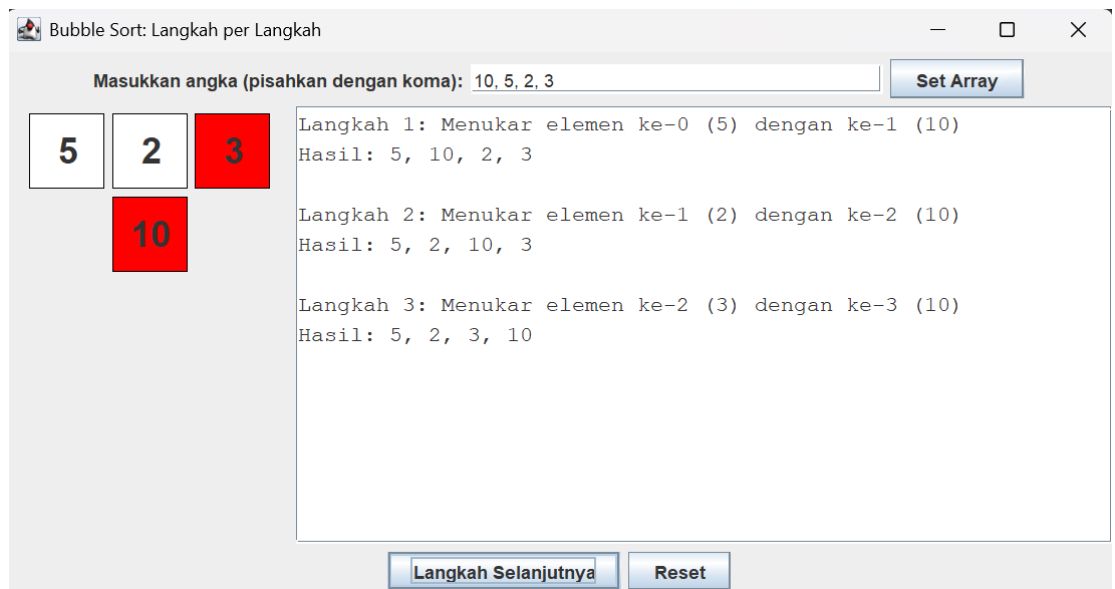
Gambar 2.3.4.2 set array



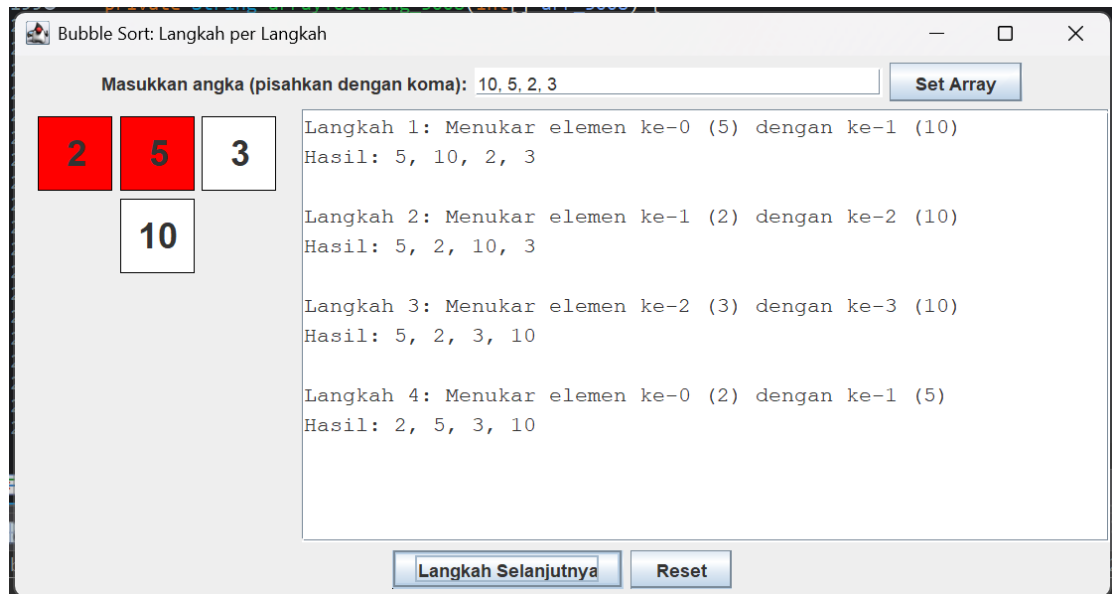
Gambar 2.3.4.3 langkah 1



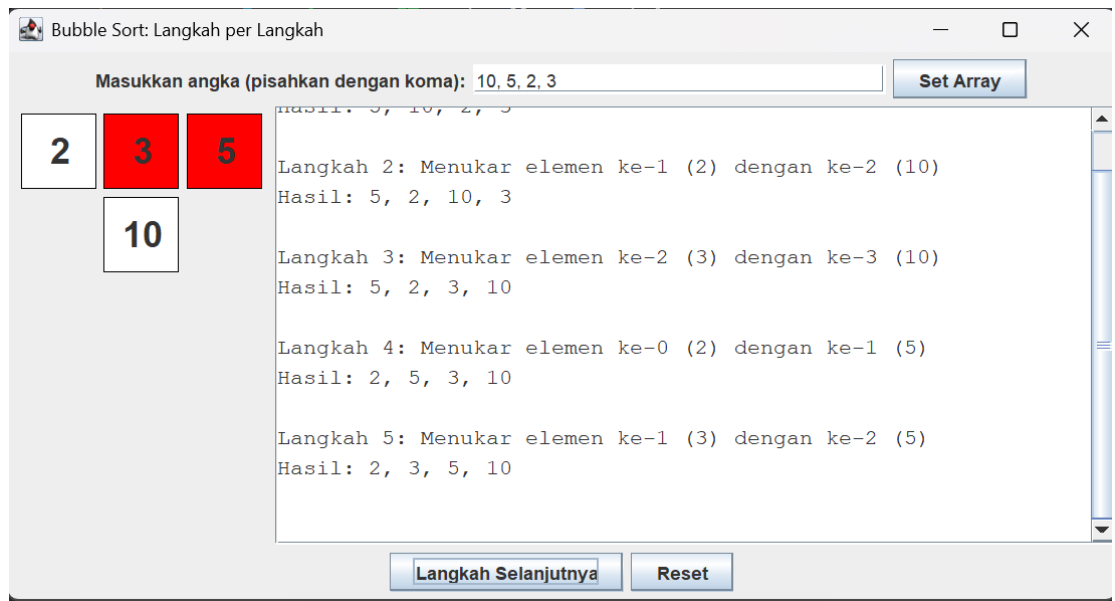
Gambar 2.3.4.4 langkah 2



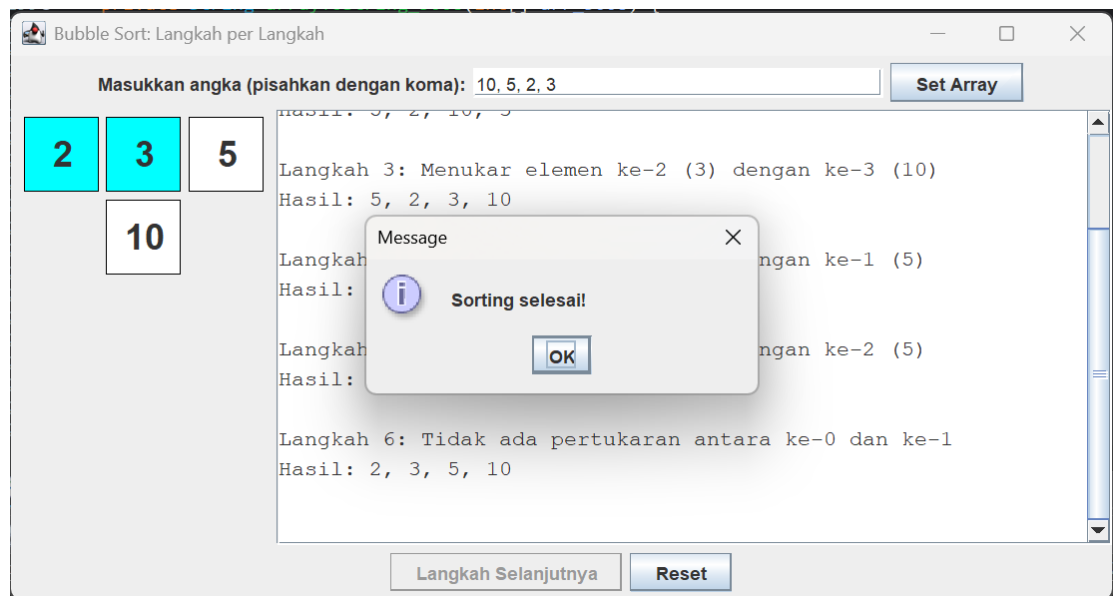
Gambar 2.3.4.5 langkah 3



Gambar 2.3.4.6 langkah 4



Gambar 2.3.4.7 langkah 5



Gambar 2.3.4.8 langkah 6 dan sorting selesai

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa algoritma sorting merupakan salah satu konsep fundamental dalam struktur data yang berfungsi untuk mengurutkan elemen-elemen data agar lebih terstruktur dan mudah diakses. Melalui implementasi algoritma seperti Bubble Sort, Shell Sort, Quick Sort, dan Merge Sort, kita dapat memahami perbedaan cara kerja masing-masing algoritma serta menganalisis efisiensi berdasarkan kompleksitas waktu. Praktikum ini juga menunjukkan bahwa pemilihan algoritma yang tepat sangat bergantung pada ukuran dan kondisi data yang diolah. Dengan menguasai konsep sorting, kita memperoleh bekal penting untuk mengembangkan algoritma lain yang lebih kompleks, meningkatkan efisiensi sistem, serta menerapkan pengurutan data dalam berbagai aplikasi nyata seperti database, sistem informasi, dan analisis statistik.

DAFTAR PUSTAKA

- [1] Universitas Andalas, , "Pertemuan 11: Sorting Algorithm Bubble and Shell," Modul Struktur Data, 2026.
- [2] Universitas Andalas, "Pertemuan 12: Sorting Algorithm," Modul Struktur Data, 2026.
- [3] M. T. Goodrich, R. Tamassia, and M. H. Goldwasser, *Data Structures and Algorithms in Java*, 6th ed. Wiley, 2014.
- [4] I. Lim, *Diktat Struktur Data*. Bandung: Institut Teknologi Bandung, 2008.

TUGAS

1. Kode Program

```
package pekan8_2511533008;

import java.util.Scanner;

class Lagu_3008 {
    String judul_3008;
    int durasi_3008; // dalam detik

    Lagu_3008(String judul_3008, int durasi_3008) {
        this.judul_3008 = judul_3008;
        this.durasi_3008 = durasi_3008;
    }

    @Override
    public String toString() {
        return judul_3008 + " - " + durasi_3008 + " detik";
    }
}

public class Sorting_2511533008 {
    Lagu_3008[] dataLagu_3008 = new Lagu_3008[20];
    int jumlahLagu_3008 = 0;

    // Input lagu
    public void inputData_3008() {
        dataLagu_3008[0] = new Lagu_3008("Rahmatun Lil'alamin", 230);
        dataLagu_3008[1] = new Lagu_3008("Qalbi Fil Madinah", 224);
        dataLagu_3008[2] = new Lagu_3008("Nas Tehbehlena", 256);
        dataLagu_3008[3] = new Lagu_3008("Sepanjang Hidup", 236);
        dataLagu_3008[4] = new Lagu_3008("Insha Allah", 270);
        dataLagu_3008[5] = new Lagu_3008("For The Rest Of My Life", 234);
        dataLagu_3008[6] = new Lagu_3008("Salla Alayka Rahman", 228);
        dataLagu_3008[7] = new Lagu_3008("Sallou", 222);
        jumlahLagu_3008 = 8;
    }

    // Quick Sort
    public void quickSort_3008(Lagu_3008[] arr_3008, int low_3008, int
high_3008) {
        if (low_3008 < high_3008) {
            int pi = partition_3008(arr_3008, low_3008, high_3008);
            quickSort_3008(arr_3008, low_3008, pi - 1);
            quickSort_3008(arr_3008, pi + 1, high_3008);
        }
    }

    private int partition_3008(Lagu_3008[] arr_3008, int low_3008, int
high_3008) {

```

```

        int pivot_3008 = arr_3008[high_3008].durasi_3008;
        int i_3008 = (low_3008 - 1);
        for (int j_3008 = low_3008; j_3008 < high_3008; j_3008++) {
            if (arr_3008[j_3008].durasi_3008 <= pivot_3008) {
                i_3008++;
                Lagu_3008 temp_3008 = arr_3008[i_3008];
                arr_3008[i_3008] = arr_3008[j_3008];
                arr_3008[j_3008] = temp_3008;
            }
        }
        Lagu_3008 temp_3008 = arr_3008[i_3008 + 1];
        arr_3008[i_3008 + 1] = arr_3008[high_3008];
        arr_3008[high_3008] = temp_3008;
        return i_3008 + 1;
    }

    // Merge Sort
    public void mergeSort_3008(Lagu_3008[] arr_3008, int left_3008, int
right_3008) {
        if (left_3008 < right_3008) {
            int mid_3008 = (left_3008 + right_3008) / 2;
            mergeSort_3008(arr_3008, left_3008, mid_3008);
            mergeSort_3008(arr_3008, mid_3008 + 1, right_3008);
            merge_3008(arr_3008, left_3008, mid_3008, right_3008);
        }
    }

    private void merge_3008(Lagu_3008[] arr_3008, int left_3008, int
mid_3008, int right_3008) {
        int n1_3008 = mid_3008 - left_3008 + 1;
        int n2_3008 = right_3008 - mid_3008;

        Lagu_3008[] L_3008 = new Lagu_3008[n1_3008];
        Lagu_3008[] R_3008 = new Lagu_3008[n2_3008];

        for (int i_3008 = 0; i_3008 < n1_3008; i_3008++) L_3008[i_3008] =
arr_3008[left_3008 + i_3008];
        for (int j_3008 = 0; j_3008 < n2_3008; j_3008++) R_3008[j_3008] =
arr_3008[mid_3008 + 1 + j_3008];

        int i_3008 = 0, j_3008 = 0, k_3008 = left_3008;
        while (i_3008 < n1_3008 && j_3008 < n2_3008) {
            if (L_3008[i_3008].durasi_3008 <= R_3008[j_3008].durasi_3008) {
                arr_3008[k_3008] = L_3008[i_3008]; i_3008++;
            } else {
                arr_3008[k_3008] = R_3008[j_3008]; j_3008++;
            }
            k_3008++;
        }
        while (i_3008 < n1_3008) { arr_3008[k_3008] = L_3008[i_3008];
i_3008++; k_3008++; }
        while (j_3008 < n2_3008) { arr_3008[k_3008] = R_3008[j_3008];
j_3008++; k_3008++; }
    }

```

```

    }

    // Shell Sort
    public void shellSort_3008(Lagu_3008[] arr_3008, int n_3008) {
        for (int gap_3008 = n_3008/2; gap_3008 > 0; gap_3008 /= 2) {
            for (int i_3008 = gap_3008; i_3008 < n_3008; i_3008++) {
                Lagu_3008 temp_3008 = arr_3008[i_3008];
                int j_3008;
                for (j_3008 = i_3008; j_3008 >= gap_3008 &&
arr_3008[j_3008-gap_3008].durasi_3008 > temp_3008.durasi_3008; j_3008 -=
gap_3008) {
                    arr_3008[j_3008] = arr_3008[j_3008-gap_3008];
                }
                arr_3008[j_3008] = temp_3008;
            }
        }
    }

    // Tampil data sebelum & sesudah sorting
    public void tampilData_3008(int pilihan_3008) {
        System.out.println("=== Sorting Playlist NIM: 2511533008 ===");
        System.out.println("Pilih Algoritma (1=Shell, 2=Quick, 3=Merge): "
+ pilihan_3008);

        System.out.println("\nData Sebelum Sorting:");
        for (int i_3008 = 0; i_3008 < jumlahLagu_3008; i_3008++) {
            System.out.println((i_3008 + 1) + ". " +
dataLagu_3008[i_3008]);
        }

        switch (pilihan_3008) {
            case 1:
                shellSort_3008(dataLagu_3008, jumlahLagu_3008);
                System.out.println("\nData Setelah Shell Sort (Durasi
Asc):");
                break;
            case 2:
                quickSort_3008(dataLagu_3008, 0, jumlahLagu_3008 - 1);
                System.out.println("\nData Setelah Quick Sort (Durasi
Asc):");
                break;
            case 3:
                mergeSort_3008(dataLagu_3008, 0, jumlahLagu_3008 - 1);
                System.out.println("\nData Setelah Merge Sort (Durasi
Asc):");
                break;
            default:
                System.out.println("\nPilihan tidak valid!");
                return;
        }

        for (int i_3008 = 0; i_3008 < jumlahLagu_3008; i_3008++) {

```

```

        System.out.println((i_3008 + 1) + ". " +
dataLagu_3008[i_3008]);
    }
}

public static void main(String[] args) {
    Sorting_2511533008 playlist_3008 = new Sorting_2511533008();
    playlist_3008.inputData_3008();

    Scanner sc_3008 = new Scanner(System.in);
    System.out.print("Masukkan pilihan algoritma (1=Shell, 2=Quick,
3=Merge): ");
    int pilihan_3008 = sc_3008.nextInt();

    playlist_3008.tampilData_3008(pilihan_3008);
}
}

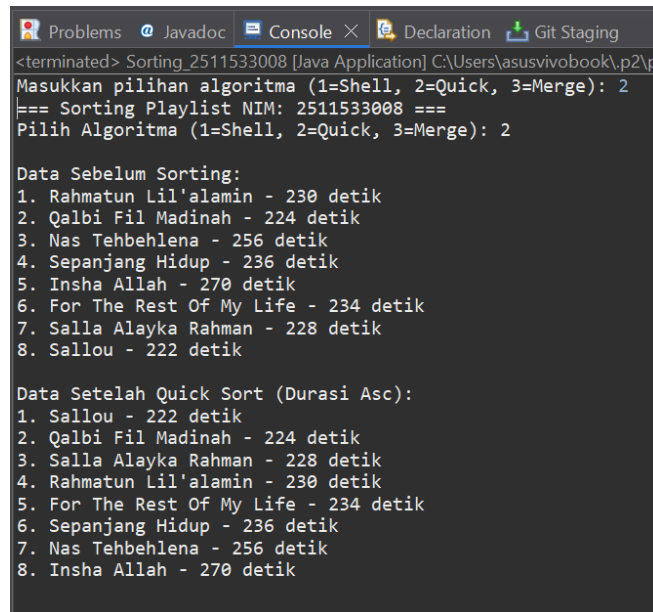
```

Penjelasan :

Merupakan gabungan dari algoritma sorting yang di pelajari dan mengaplikasikannya untuk mengurutkan lagu berdasarkan durasi.

Untuk mengurutkan lagu berdasarkan durasi disarankan untuk menggunakan algoritma quick sort karena lebih cocok untuk mengurutkan angka.

2. Output Program



```

<terminated> Sorting_2511533008 [Java Application] C:\Users\asusvivobook\p2\p
Masukkan pilihan algoritma (1=Shell, 2=Quick, 3=Merge): 2
=== Sorting Playlist NIM: 2511533008 ===
Pilih Algoritma (1=Shell, 2=Quick, 3=Merge): 2

Data Sebelum Sorting:
1. Rahmatun Lil'alamin - 230 detik
2. Qalbi Fil Madinah - 224 detik
3. Nas Tehbehlena - 256 detik
4. Sepanjang Hidup - 236 detik
5. Insha Allah - 270 detik
6. For The Rest Of My Life - 234 detik
7. Salla Alayka Rahman - 228 detik
8. Sallou - 222 detik

Data Setelah Quick Sort (Durasi Asc):
1. Sallou - 222 detik
2. Qalbi Fil Madinah - 224 detik
3. Salla Alayka Rahman - 228 detik
4. Rahmatun Lil'alamin - 230 detik
5. For The Rest Of My Life - 234 detik
6. Sepanjang Hidup - 236 detik
7. Nas Tehbehlena - 256 detik
8. Insha Allah - 270 detik

```

Gambar 2. Output tugas